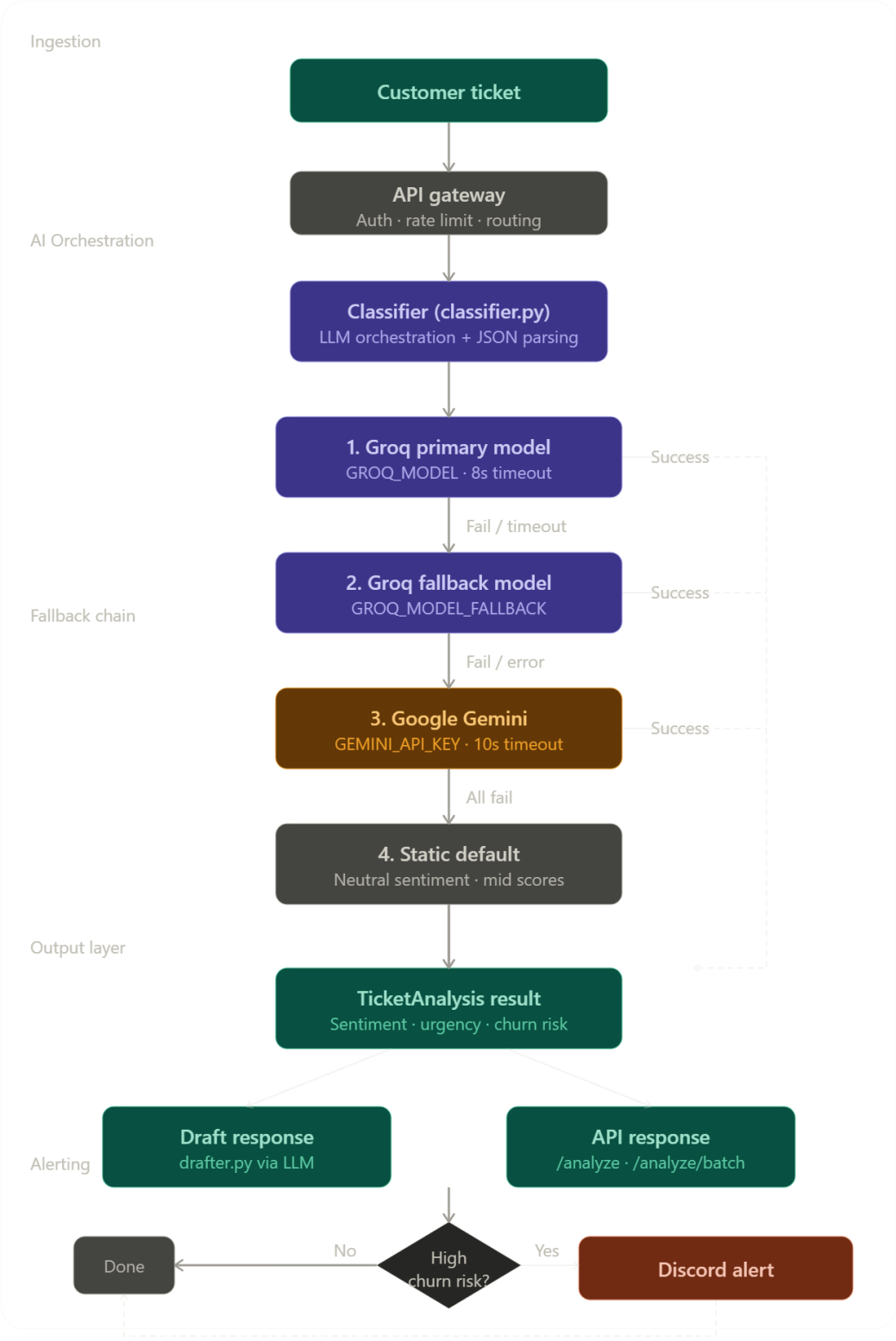
FLOW CHART


Ingestion
Customer ticket
API gateway
Auth · rate limit · routing
AI Orchestration
Classifier (classifier.py)
LLM orchestration + JSON parsing
1. Groq primary model
GROQ_MODEL · 8s timeout
Success
Fail / timeout
2. Groq fallback model
GROQ_MODEL_FALLBACK
Success
Fallback chain
Fail / error
3. Google Gemini
GEMINI_API_KEY · 10s timeout
Success
All fail
4. Static default
Neutral sentiment · mid scores
Output layer
TicketAnalysis result
Sentiment · urgency · churn risk
Alerting
Draft response
drafter.py via LLM
API response
/analyze · /analyze/batch
High churn risk?
No
Yes
Done
Discord alert

Ingestion — a customer ticket hits the API gateway (main.py), which handles auth and rate limiting before passing it to the classifier.

AI Orchestration — classifier.py acts as the LLM router. It builds the prompt, calls the model, and handles JSON parsing (including stripping fences and regex recovery before triggering any fallback).

Fallback chain — a strict 4-step cascade:

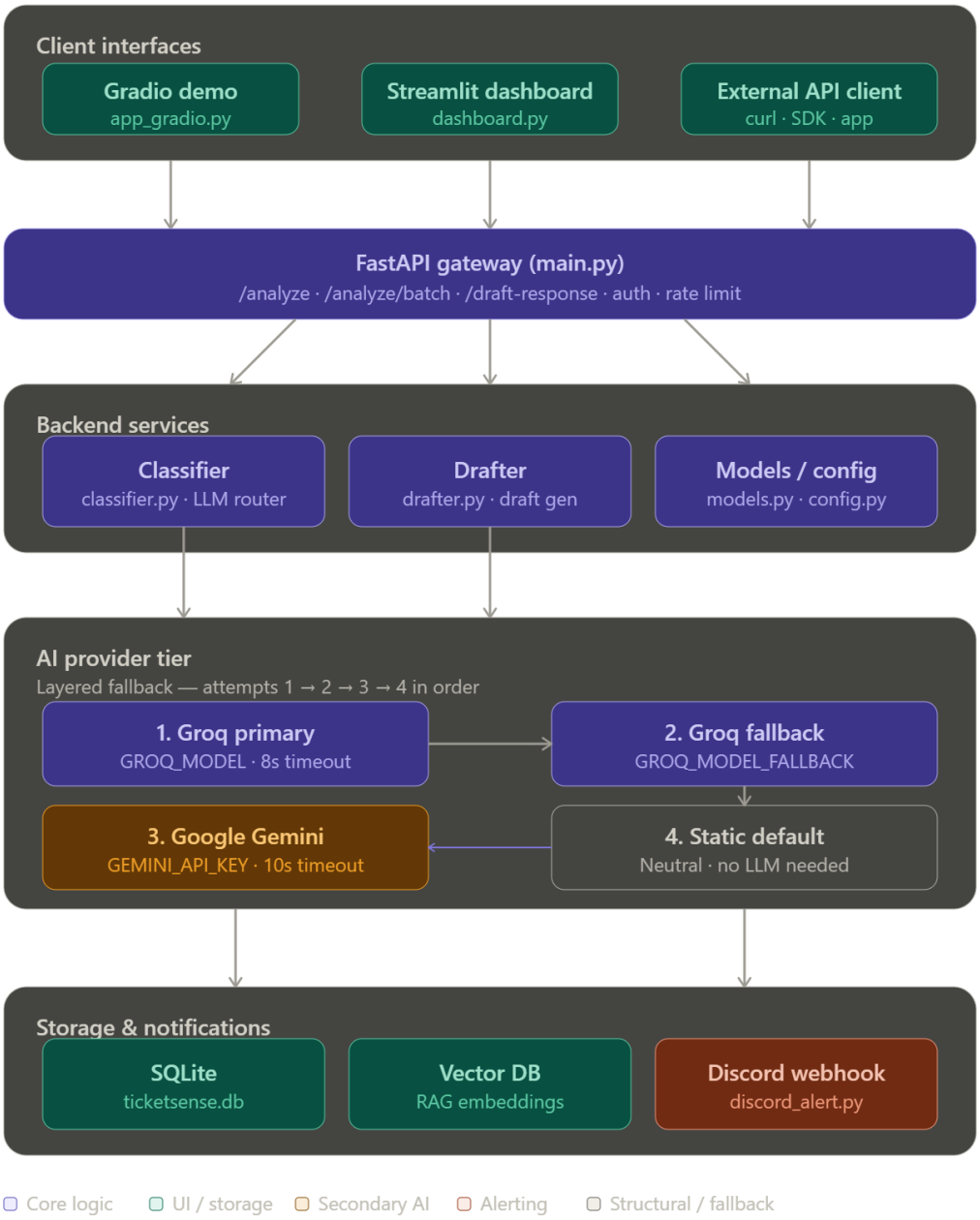
1. Primary Groq model (8s timeout)
2. Groq fallback model
3. Google Gemini (10s timeout)
4. Safe static TicketAnalysis default — ensures downstream systems are never blocked

The dashed rail on the right shows successful responses bypassing lower rungs and feeding directly into the output layer.

Output layer — a TicketAnalysis object (sentiment, urgency, churn risk) is produced. From there, drafter.py generates a draft response in parallel with the API response going back to the caller.

Alerting — a churn-risk check gates whether a Discord webhook fires. Low-risk tickets exit quietly; high-risk ones trigger discord_alert.py

ARCHITECTURE DIAGRAM



Client interfaces — three entry points: the Gradio demo for quick testing, the Streamlit ops dashboard, and any external API caller hitting the REST endpoints directly.

API gateway — a single FastAPI app (main.py) fronts everything, handling auth, rate limiting, and routing to the three main endpoints.

Backend services — the classifier (LLM orchestration + JSON parsing), the drafter (response generation), and the Pydantic models/config layer that glues them together.

AI provider tier — the 4-step fallback chain lives entirely within this boundary. Purple arrows show the cascade: Groq primary → Groq fallback → Gemini → static default.

Storage & notifications — SQLite for persistence, a vector DB for RAG-powered context, and the Discord webhook for high-churn-risk alerts.